

LocalMind OS

Project Overview, Architecture, and End-to-End Workflow

This document explains what the project does, how the frontend and backend work together, and what happens step by step when a user uploads a PDF, searches it, or asks a grounded question.

What this PDF covers

- What LocalMind OS is and why it exists
- How the application is structured
- What happens when a PDF is uploaded
- How chunking, embeddings, and indexing work
- How search and trust-mode chat produce answers
- Which models and fallbacks the app uses today

2. What the Project Is

LocalMind OS is a local-first knowledge workspace. It lets a user upload private documents, turn them into searchable chunks, ask grounded questions, and explore related topics without sending the data to a cloud service by default.

The frontend is a Next.js app, while the backend is a FastAPI service. The backend owns the document pipeline, search pipeline, knowledge graph, security vault, and model selection.

Layer	Role
Frontend	Upload, search, chat, model controls, graph and evaluation views
Backend	Ingestion, chunking, embeddings, retrieval, RAG, security, persistence
Storage	Encrypted runtime data under backend/data and uploaded files under backend/data/uploads
Models	Local embeddings, optional reranker, local GGUF LLM or extractive fallback

The core idea is simple: documents come in, are transformed into smaller meaning-rich pieces, and those pieces are made available for search and answer generation.

3. End-to-End Workflow

The app follows a consistent pipeline. First, a file is uploaded. Then the backend extracts text, chunks the content, embeds each chunk, stores the vectors, and updates the graph and metadata. After that, the data becomes available for search, chat, and analytics.

- User selects files in the frontend.
- Frontend calls POST /ingest with the files.
- Backend stores uploads in backend/data/uploads.
- Text is extracted from PDFs or other supported formats.
- `chunk_document()` splits content into structured chunks.
- `EmbeddingService` converts chunks into vectors.
- `VectorIndex` stores those vectors in FAISS or NumPy fallback.
- Search and chat use the indexed chunks as evidence.

This design keeps the system offline-friendly and makes every stage inspectable. The same uploaded source can be reused for semantic search, grounded chat, and graph generation.

4. What Happens When You Upload a PDF

When a PDF is uploaded, the backend first validates that the app is unlocked. The file is saved to disk and a background ingestion job is created so the frontend can keep polling job status.

- The upload endpoint is POST /ingest.
- Files are stored with their original names under the uploads directory.
- The job status endpoint lets the UI show progress.
- Duplicate source files are skipped during ingestion if already indexed.
- The system can also ingest built-in demo files through /ingest_demo.

Concrete example

Suppose you upload **ML_Loss_Comparison.pdf**. The backend keeps the original PDF, extracts text page by page, and then passes the text to the chunking pipeline. The file then becomes searchable and usable in chat without any manual labeling.

5. How Chunking Is Done

Chunking is the step that turns large documents into smaller retrieval units. In this project, the backend uses a structured chunker with heading awareness and overlap. The current chunking version is **structured-v3**.

- The backend calls `chunk_document(doc.text, chunk_size=900, overlap=150)`.
- Headings are preserved as a section path, such as Section: Chapter 2 > Loss Functions.
- Paragraphs, lists, and long sentences are split differently so chunks stay readable.
- Adjacent chunks share overlap so context is not lost at boundaries.

Example chunking flow

If a PDF page contains a section called **Loss Functions** with three paragraphs, the first chunk may contain the heading plus the first paragraph, and the second chunk may reuse the last part of the first chunk as overlap. That means a query about the transition between two ideas can still be answered from retrieval.

This is important because retrieval works better when each chunk is a self-contained evidence unit instead of a raw page dump.

6. Which Models the App Uses

The app uses a layered model strategy rather than a single fixed model. The selected model depends on what is available on the machine and what the user has configured in the model manager.

Component	Current behavior
LLM	Defaults to extractive-fallback at startup; can switch to a local GGUF model via llama-cpp-python
Embeddings	Tries sentence-transformers/all-MiniLM-L6-v2 or local embedding folders first
Embedding fallback	Uses hashed TF-IDF if no sentence-transformer model loads
Vector index	Uses FAISS when installed, otherwise NumPy similarity fallback
Reranker	Optional local cross-encoder; otherwise lexical reranking only

In practice, the default setup is intentionally safe for a laptop: the backend does not require a cloud API key, and it can still function even when a higher-end local model is not installed.

7. How Indexing and Search Work

After chunking, the embedding service converts each chunk into a numerical vector. The vector index stores those vectors so the app can find similar chunks when the user searches.

- EmbeddingService first tries a sentence-transformers model.
- If that is unavailable, it falls back to hashed TF-IDF.
- VectorIndex stores the embeddings in FAISS or in-memory NumPy similarity.
- Search combines vector similarity with lexical scoring and diversity selection.
- A reranker can improve ordering when a local cross-encoder is installed.

Search example

If the user searches for **How does cross entropy compare to hinge loss?**, the backend builds an embedding for that question, retrieves the nearest chunks, adds lexical variants, and then reranks the results so the most relevant chunk appears first.

8. How Grounded Chat Produces an Answer

The ask endpoint uses the search pipeline as its evidence source. It retrieves chunks, evaluates how strong the evidence is, and then decides whether to answer directly, fall back to extractive answering, or refuse weakly grounded requests in trust mode.

- The frontend sends a question to POST /ask.
- The backend optionally follows chat history for follow-up questions.
- Relevant chunks become cited sources like S1, S2, and S3.
- If evidence is weak, trust mode can return a refusal or a short extractive answer.
- If a GGUF model is available, `rag_engine.generate_answer()` can produce the final response.

Why this matters

This keeps answers grounded in the uploaded documents instead of letting the model invent details. The chat experience is therefore tied to the actual corpus you uploaded.

9. Worked Example

Here is a complete example from upload to answer generation.

Step-by-step example

- You upload a PDF about machine learning losses.
- The backend extracts text and splits it into structured chunks.
- A chunk may look like: Section: Loss Functions > Hinge Loss ...
- The embedding model turns that chunk into a vector.
- The index stores the vector and metadata such as source file and page number.
- You ask: What is the difference between hinge loss and cross entropy?
- Search retrieves the most relevant chunks and chat uses them as sources.

Example output shape

The answer may include a direct explanation, a short comparison, and citations like [S1] and [S2]. If the question is a study task, the same sources can be reformatted into a guide, flashcards, or a quiz.

10. Security, Storage, and Closing Notes

LocalMind OS is designed to keep data local. The backend stores uploads, chunks, graph data, query logs, and model settings under backend/data, and the vault layer encrypts persisted runtime artifacts when the passphrase is configured.

- First launch asks the user to create a passphrase.
- Later launches require the same passphrase to unlock the backend.
- If the passphrase is lost, encrypted runtime data cannot be recovered through the app.
- The app still works in fallback modes even when high-end local models are not installed.

Summary

The project is a local knowledge workspace built around a reliable retrieval pipeline: upload, extract, chunk, embed, index, retrieve, and answer with evidence. That is the full loop the user sees in the product.

If you add stronger local models later, the architecture already supports them without changing the basic workflow.